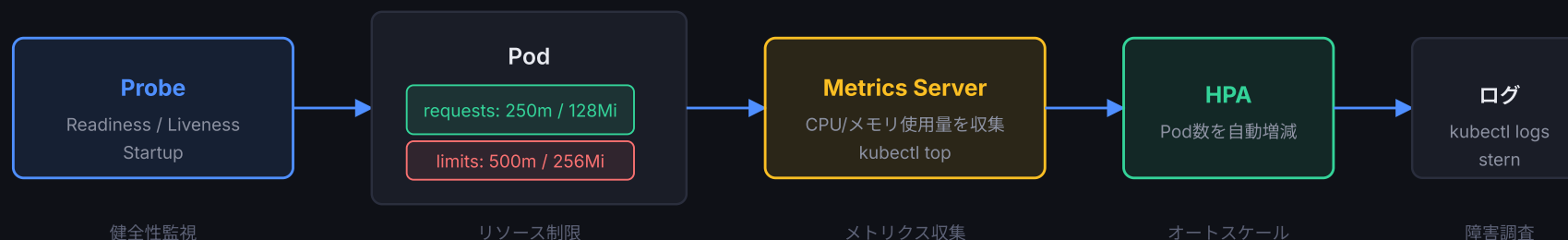


Kubernetes Foundation

Kubernetesの運用監視 Probe ・ HPA ・ ログ

自律回復 / オートスケール / ログ調査

Probe・requests/limits・HPA・ログ収集の4本柱でK8sアプリを自律回復・自動スケールする



01

Probeで自律回復する

Readiness/Liveness/StartupProbeがコンテナの状態を定期チェック。異常Podをトラフィックから外し、自動再起動する

02

requests/limits+HPAで自動スケール

requestsはスケジューリング基準・limitsは暴走防止。Metrics ServerがCPU使用率を取得しHPAがPod数を自動増減する

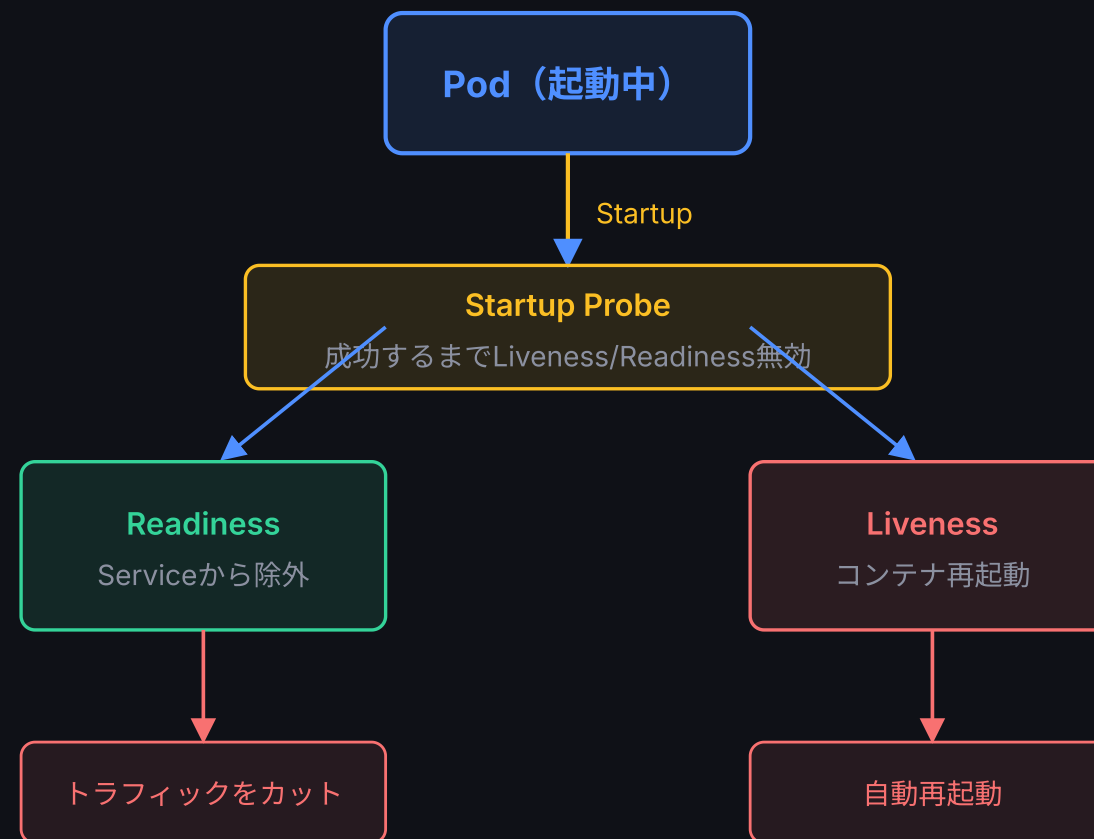
03

ログ力がKubernetes運用力

stdoutはノードのファイルに集約される。kubectl logs/sternで複数Podのログを横断調査。本番はFluentdでCloudWatchへ転送

Probeの3種類がコンテナの健全性を保つ

- **Readiness Probe**：「リクエストを受け付けられるか」をチェック。**NGならServiceから外す**。DBへの接続待ちなど起動直後の保護
- **Liveness Probe**：「コンテナが活着ているか」をチェック。**NGならコンテナを再起動**。ハングアップ・メモリリークの自動回復
- **Startup Probe**：起動に時間がかかるアプリ向け。**成功するまでLiveness/Readinessを無効化**。誤判定による再起動を防ぐ



requests/limitsでリソースを安全に管理する

Node (CPU: 4コア / Memory: 8GB)

CPU

requests: 250m

+limits分

Mem

requests: 128Mi

limits: 256Mi

```
resources:
```

```
  requests:
```

```
    cpu: "250m" # スケジューリング基準
```

```
    memory: "128Mi"
```

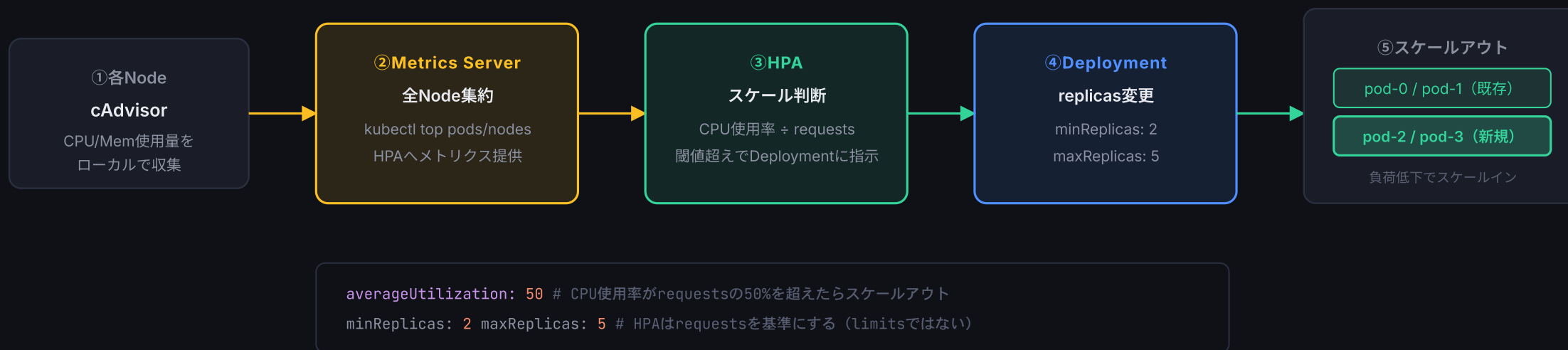
```
  limits:
```

```
    cpu: "500m" # OOM Kill / CPU throttle
```

- **requests**：最低限確保するリソース量。**スケジューリングの判断基準**。requestsを元にNodeを選ぶ
- **limits**：使ってよい上限。**CPUは絞られ、メモリ超過はOOM Kill**。ノイジーネイバー問題を防ぐ
- **HPAの基準もrequests**：CPU使用率= 実使用量 ÷ requests × 100%。limitsは無関係なので注意

設定なしのリスク: requestsゼロ→Nodeに過剰配置、limitsなし→Pod暴走でNode障害

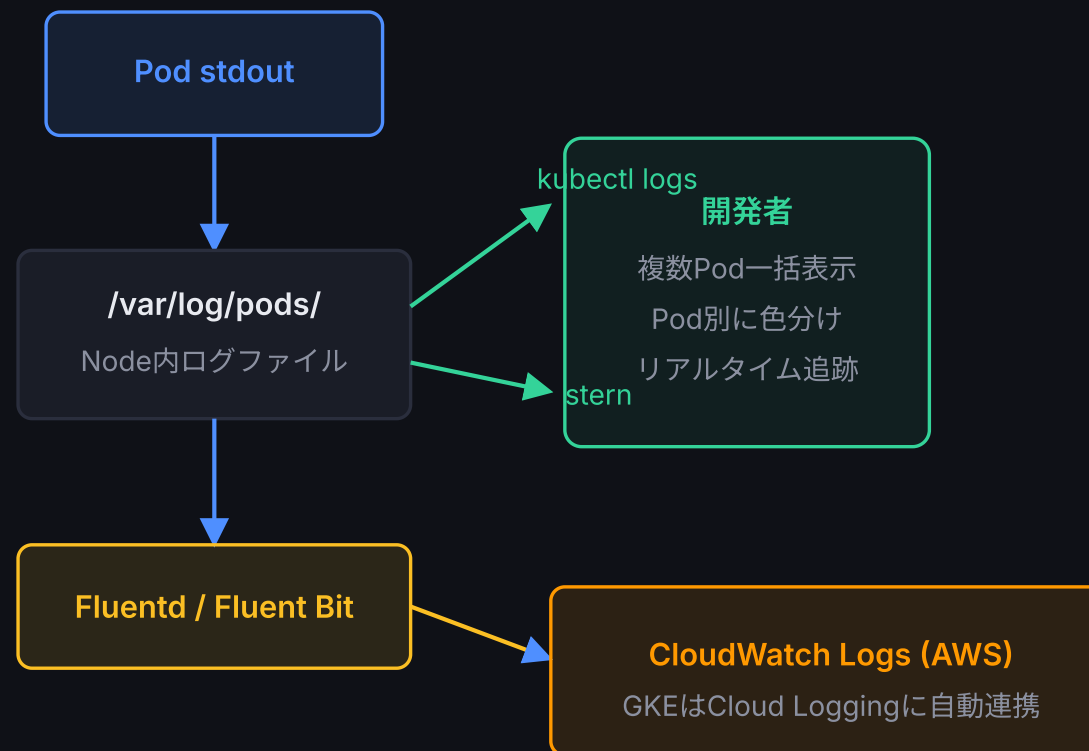
Metrics Server → HPA → スケールアウトの流れ



kubectl logsとsternでログを調査する

- PodのstdoutはNode内の/var/log/pods/に自動収集。
kubectlはこのファイルを読んで表示している
- **kubectl logs <pod>** : 1Pod単位。 **stern <prefix>** を使えば
名前に部分一致する複数PodのログをPod別色分けで一括表示
- 本番EKSでは**Fluentd/Fluent Bit**をDaemonSetで動かして
CloudWatch Logsへ転送。GKEはCloud Loggingに自動連携

ログを読む力 = Kubernetes運用力。障害調査は必ずPodのログから始まる



Readiness / Liveness / Startup Probe の使い分け

PROBE	チェック内容	失敗時のアクション	主な使用場面
Readiness	リクエストを受け付けられるか	Serviceのエンドポイントから除外	起動直後のDB接続待ち / デプロイ時の保護
Liveness	コンテナプロセスが活着ているか	コンテナを自動再起動	ハングアップ / メモリリーク / 無限ループ検知
Startup	起動完了したか（猶予期間）	成功まで他ProbeはSKIP / 失敗で再起動	起動時間が変動するアプリ（Java等）の誤判定防止

本番では **Readiness + Liveness の2つをセットで設定** することが推奨。Startup Probeは起動時間がブレる場合に追加する。

まとめ

01

ProbeとrequestsはセットでON

Readiness+LivenessProbeで自律回復。requests/limitsを必ず設定してスケジューリング精度を上げ、Nodeの過負荷を防ぐ

02

HPAにはMetrics Serverが必要

EKSでは手動インストール必須。HPAはrequestsを基準にCPU使用率を計算するため、requestsの設定がHPA精度に直結する

03

ログ調査はsternから始める

kubectl logsは1Pod単位、sternで複数Pod横断。本番はFluentdでCloudWatch/Cloud Loggingへ集約。ログ力が運用力